DBT • CLASS NOTES

Stored Functions & Triggers

DBT Day 8 — 16 October

Complete, visual notes from today's lecture: writing your own stored functions, the binary-logging error (and its fixes), the six trigger events, OLD vs NEW, the salary-audit and stock-management triggers, and how triggers behave inside a transaction — with every query ready to run.

LEARN BY DOING

Study notes • prepared 16 October 2026
Topics: stored functions • triggers | MySQL 8.0

Table of Contents

Day 8 at a Glance 📌
- 🎯 What we covered today
- 🗃️ The tables we used in class
- 🧭 The shape of today

Part A — Stored Functions 🧪
- 🔧 What is a stored function?
- 🧱 The syntax, piece by piece
- 🥇 Example 1 — a function with no parameters
- 🥈 Example 2 — a function with one parameter
- ⚠️ The error that stopped everyone today
- 🥉 Example 3 — a function with multiple parameters
- 🌳 Example 4 — making decisions inside a function: getGrade
- 👀 Seeing and managing your functions
- 🎯 Where you can use a function
- 💼 Practical example — the annual package
- 🏷️ Practical example — a discount function
- 📝 Variables inside a function
- 📌 One more thing: the characteristics
- ✅ Part A recap

Part B — Triggers ⚡
- ⚡ What is a trigger?
- 🎛️ The six events you can trigger on
- 🔄 OLD and NEW — what is available when
- 🧱 The general syntax
- 📝 Example 1 — the salary audit (AFTER UPDATE)
- 🚫 Example 2 — validation with SIGNAL (BEFORE INSERT)
- 📦 Example 3 — stock management (AFTER INSERT on orders)
- 🚧 The "same table" rule (error 1442)
- 🔢 Multiple triggers on the same event: FOLLOWS and PRECEDES
- 🧭 What triggers do not do
- ↩️ Triggers, transactions and ROLLBACK — the class demo
- 🛠️ Managing triggers
- ✅ Part B recap

Part C — Cheat Sheet, Gotchas & Practice 🏁
- 📋 Cheat sheet
- 🐞 Errors we actually hit today — and exactly how to fix them
- 🧠 Practice exercises (do these — they mirror the class)
- 🧩 Solutions
- ➡️ Next up
- 🎓 Final words

Day 8 at a Glance 📌

These are your notes from **DBT Day 8 (16 Oct)** — the session that picked up right after stored procedures and covered **two big topics**: **stored functions** and **triggers**. This chapter is the map for everything that follows.

> 💡 **How to read these notes**
>
> Keep MySQL open next to this PDF. Every concept is followed by a query you can **type and run yourself**. Type it — do not copy-paste — and try to *predict* the output before you press `Enter`. That is how a lecture turns into a skill.

🎯 What we covered today

Part	Topic	The one-line summary
A	Stored **Functions**	A named program that **must return one value**, called with `SELECT`
B	**Triggers**	Code that fires **by itself** when a table is `INSERT`ed / `UPDATE`d / `DELETE`d

We also spent real time on the **error that stopped almost everyone** (the binary-logging / `DETERMINISTIC` error) and on **how triggers behave inside a transaction** when you `ROLLBACK`. Both get their own sections.

> ℹ️ **Where we are in the course**
>
> - **Yesterday:** stored **procedures** — reusable programs you call with `CALL`.
> - **Today:** stored **functions** (must return a value) and **triggers** (fire automatically).
> - **Next class: cursors** and **error handling** 🎞️

🗂️ The tables we used in class

We reused the `student` and `employee` tables from earlier days and created a few new ones today. Run this once so every example in these notes works.

```sql
-- ------------------------------------------------------------------------
-- Tables we already had (from earlier days) - shown here for reference
-- ------------------------------------------------------------------------
--   student  (id, name, marks)                -> used for getGrade()
--   employee (emp_id, emp_name, salary)       -> used for audit + validation
-- ------------------------------------------------------------------------

-- ------------------------------------------------------------------------
-- Tables we created TODAY
-- ------------------------------------------------------------------------

-- The product + orders tables for the stock-management trigger
CREATE TABLE product (
  product_id   INT PRIMARY KEY,
  product_name VARCHAR(30),
  stock        INT
);

-- NOTE: the table is called "orders", NOT "order" - "ORDER" is a reserved
-- keyword in MySQL and cannot be used as a table name (we hit this in class!)
CREATE TABLE orders (
  order_id   INT,
  product_id INT,
  quantity   INT
);

-- The audit table for the salary-audit trigger
CREATE TABLE salary_audit (
  audit_id   INT AUTO_INCREMENT PRIMARY KEY,
  emp_id     INT,
  emp_name   VARCHAR(30),
  old_salary INT,
  new_salary INT,
  change_on  DATETIME
);

-- A "replica" table we used to work around the same-table rule
CREATE TABLE employee_replica (
  emp_id   INT,
  emp_name VARCHAR(30),
  salary   INT
);
```

Some seed data

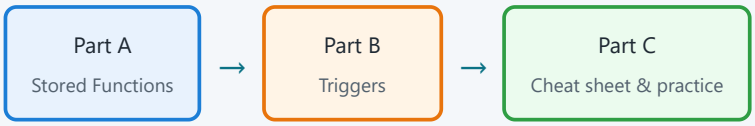
```sql
INSERT INTO product (product_id, product_name, stock) VALUES
  (1, 'iPhone 17',   300),
  (2, 'Samsung Pro', 250),
  (3, 'Nokia',       500);

INSERT INTO employee (emp_id, emp_name, salary) VALUES
  (101, 'Amit',  80000),
  (102, 'Priya', 50000),
  (103, 'Rakesh', 60000);

INSERT INTO student (id, name, marks) VALUES
  (1, 'Aarav', 95),
  (2, 'Diya',  82),
  (3, 'Rohan', 68),
  (4, 'Meera', 45);
```

> ⚠️ **Table names may differ on your machine**
>
> If your `student` / `employee` tables use different column names, adjust the queries. The *ideas* are what matter, and they are identical either way.

🧭 The shape of today



Two topics, one habit: run every example yourself.

Ready? Let's teach the database a new verb. 💉

Part A — Stored Functions 🧪

You already met the **built-in** functions (`CONCAT` , `ROUND` , `NOW` ...). Today you learned to write your **own** — programs that live inside MySQL and behave like new SQL keywords.

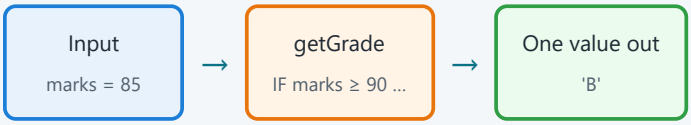
🔧 What is a stored function?

A **stored function** is a **named program inside MySQL** that:

1. **takes some input** (parameters — possibly none),
2. **performs a task** (some logic), and
3. **must return exactly one value**.

> ℹ️ **The analogy from class**
>
> In Java, a method has a **return type**. A function that produces a value returns, say, an `int` ; a `void` method returns nothing. In MySQL the same idea applies: a **function always returns a value**, and you declare its type with `RETURNS` .

A function is a machine 🔩



Put something in, get exactly one thing out — every single time.

Function vs Procedure — the difference the teacher stressed

	Stored Procedure	Stored Function
Must return a value?	No (optional)	**Yes — always**
How you call it	`CALL proc(...)`	`SELECT fn(...)`
Can it be used inside a `SELECT` ?	❌ No	✅ Yes
Parameter directions	`IN` , `OUT` , `INOUT`	`IN` only
Can it return a result set (table)?	✅ Yes	❌ No — only one value
Typical job	Perform a task	Compute a value

> 💡 **Remember it in one line**
>
> **Procedure = do a task. Function = give me a value.**

🧱 The syntax, piece by piece

Here is the skeleton. Read the comments — every line is explained below.

```
DELIMITER $$                        -- (1) change the statement separator

CREATE FUNCTION fn_name (p1 INT, p2 VARCHAR(20))   -- (2) name + parameters
RETURNS INT                         -- (3) the type of the single value you return
DETERMINISTIC                       -- (4) a promise to MySQL (explained later)
BEGIN                               -- (5) start of the body
    -- your logic goes here
    RETURN something;               -- (6) hand the value back
END $$                              -- (7) end of the body

DELIMITER ;                         -- (8) put the normal separator back
```

#	Piece	What it does
1	`DELIMITER $$`	The body contains `;` characters. This tells the client "the statement ends at `$$`, not `;`".
2	`CREATE FUNCTION fn_name(...)`	Names the function and lists its inputs.
3	`RETURNS <type>`	**Mandatory.** The data type of the single value you will return.
4	`DETERMINISTIC` / `NO SQL` / `READS SQL DATA`	Tells MySQL how the function behaves (this is what fixed today's big error).
5–7	`BEGIN … END`	The body. Use `RETURN` to send a value back.
8	`DELIMITER ;`	Back to normal so your next `;` works as expected.

> ⚠️ **Why the `DELIMITER` dance?**
>
> Inside the body there are internal semicolons. Without changing the delimiter, MySQL would think the function ends at the **first** internal `;`. Changing it to `$$` (or `//`) keeps the whole body together. This is exactly the same reason we used `DELIMITER` for stored procedures.

🥇 Example 1 — a function with no parameters

The simplest possible function: it takes no input and always returns the same value.

```
DELIMITER $$
CREATE FUNCTION getCentre()
RETURNS VARCHAR(50)
DETERMINISTIC
RETURN 'SEDAC Mumbai';
$$
DELIMITER ;
```

Call it with `SELECT`:

```
SELECT getCentre();
```

Expected output

```
+---------------+
| getCentre()   |
+---------------+
| SEDAC Mumbai  |
+---------------+
```

> **No `CALL` for functions**
>
> In class we confirmed this: **you call a procedure with `CALL`, but you call a function with `SELECT`.** The function appears as if it were a column.

Example 2 — a function with one parameter

The class example: give it a number, get its square back.

```
DELIMITER $$
CREATE FUNCTION square_number(n INT)
RETURNS INT
DETERMINISTIC
RETURN n * n;
$$
DELIMITER ;

SELECT square_number(7);     -- 49
SELECT square_number(12);    -- 144
```

Expected output

```
+------------------+
| square_number(7) |
+------------------+
|               49 |
+------------------+
```

⚠️ The error that stopped everyone today

This was the big event of the lecture. When many of you created a function, MySQL refused with this message:

```
ERROR 1418 (HY000): This function has none of DETERMINISTIC, NO SQL, or READS SQL
DATA in its declaration and binary logging is enabled (you *might* want to use the
less safe log_bin_trust_function_creators variable)
```

Why it happens 🤔

MySQL keeps a **binary log** — a record of every change, used for replication and point-in-time recovery. When you create a function, MySQL wants to be sure the function is **safe to replay** on another server. If your function says nothing about its behaviour, MySQL plays it safe and **refuses to create it**.

Two ways out — pick one



Fix 1 is the correct habit. Fix 2 is a quick unblock while you are learning.

Fix 1 — tell MySQL how the function behaves (recommended). Add one of these right after `RETURNS <type>`:

```
CREATE FUNCTION square_number(n INT)
RETURNS INT
DETERMINISTIC        -- <- add this line  (same inputs always give same output)
RETURN n * n;
```

Characteristic	Add it when…
`DETERMINISTIC`	the function always returns the **same output for the same input** (a pure calculation)
`NO SQL`	the body contains **no SQL statements at all**
`READS SQL DATA`	the body **reads** from tables but does not change them

Fix 2 — turn the safety check off (for your practice machine only).

```
SET GLOBAL log_bin_trust_function_creators = 1;

-- verify it:
SHOW VARIABLES LIKE 'log_bin_trust_function_creators';
```

> 🛑 **Still got an error after the first fix? Check the spelling!**
>
> In class the same error came back for a second reason: the keyword was typed as `READ SQL DATA`. It must be `READS SQL DATA` — with an **s**. `READS` (plural). MySQL is unforgiving about this.

> ⚠️ **About `SET GLOBAL`**
>
> `SET GLOBAL` affects the whole server and **resets when MySQL restarts** unless you put it in the configuration file. It is fine while learning, but in production the right answer is **Fix 1** — declare the function honestly.

🥉 Example 3 — a function with multiple parameters

The class example: given a **price** and a **quantity**, return the **total**.

```
DELIMITER $$
CREATE FUNCTION total_price(price DECIMAL(10,2), quantity INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
RETURN price * quantity;
$$
DELIMITER ;

SELECT total_price(281.00, 2);      -- 562.00
```

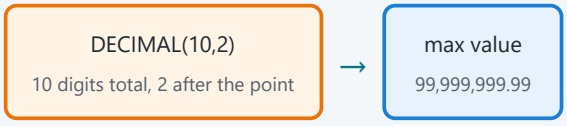
Expected output

```
+-------------------------+
| total_price(281.00, 2)  |
+-------------------------+
|                  562.00 |
+-------------------------+
```

Why `DECIMAL(10,2)` and why return `DECIMAL(12,2)`? 💰

A question came up in class about the `DECIMAL(10,2)`. Here is the full answer.

`DECIMAL(M, D)` means: **M total digits**, of which **D are after the decimal point**.



DECIMAL(10,2) stores money exactly — no rounding surprises.

Type	Exact?	Use it for
`DECIMAL(M,D)`	✅ exact	**money** (prices, salaries)
`FLOAT` / `DOUBLE`	❌ approximate	scientific numbers where tiny errors don't matter

> 💉 **Why the return type was widened to (12,2)**
>
> Price × quantity can be **bigger** than a single price. If the return type is too small for the result, MySQL either errors with "*Out of range value for column*" or clamps/truncates the value. Giving the result **two extra digits** (`12,2` instead of `10,2`) is a safety margin so a large total never overflows.

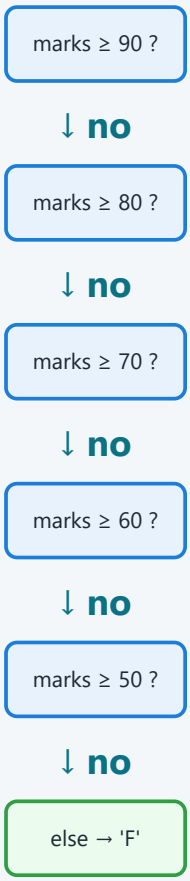
> ⚠️ **Return-type capacity matters**
>
> If you ever see "*Out of range value*" when calling a function, the fix is usually to **widen the** `RETURNS` **type** (more digits) — exactly what the teacher did here.

🌳 Example 4 — making decisions inside a function: `getGrade`

This is the most useful function from today: turn **marks** into a **grade** using an `IF / ELSEIF / ELSE` ladder.

```
DELIMITER $$
CREATE FUNCTION getGrade(marks INT)
RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
    IF marks >= 90 THEN
        RETURN 'A+';
    ELSEIF marks >= 80 THEN
        RETURN 'A';
    ELSEIF marks >= 70 THEN
        RETURN 'B';
    ELSEIF marks >= 60 THEN
        RETURN 'C';
    ELSEIF marks >= 50 THEN
        RETURN 'D';
    ELSE
        RETURN 'F';
    END IF;
END $$
DELIMITER ;
```

The decision ladder 🪜



MySQL checks top-down and stops at the first TRUE. If none match, ELSE catches it.

Each `RETURN` **exits the function immediately** — so the moment a condition matches, MySQL sends that grade back and stops reading the rest.

> 💡 **Test the boundaries, not just the middle**
>
> ```
> SELECT getGrade(95),   -- A+
>        getGrade(90),   -- A+   (the >= boundary)
>        getGrade(89),   -- A
>        getGrade(50),   -- D    (the >= boundary)
>        getGrade(49);   -- F
> ```
>
> Boundary tests are how you catch off-by-one mistakes. Always test the edge.

> ℹ️ **Why `RETURNS VARCHAR(2)` ?**
>
> Because the longest result is `'A+'` — two characters. Sizing the return type to the largest actual value keeps your data tidy.

👀 Seeing and managing your functions

A student asked how to list the functions they had created. Here are all the commands:

```
-- List every function in the CURRENT database (status table)
SHOW FUNCTION STATUS WHERE Db = DATABASE();

-- See the exact SQL that created one function
SHOW CREATE FUNCTION getGrade;

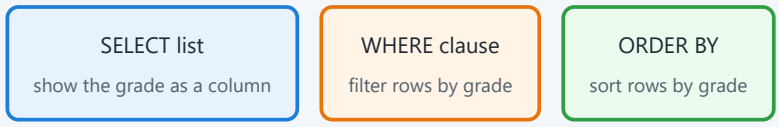
-- See the definition of a PROCEDURE in the same way
SHOW CREATE PROCEDURE my_procedure;

-- Delete a function
DROP FUNCTION IF EXISTS getGrade;
```

> 💡 **`SHOW CREATE` is your debugging best friend**
>
> `SHOW CREATE FUNCTION getGrade;` prints back the whole definition — perfect when a function exists but "doesn't do what you think it does".

🎯 Where you can use a function

A function returns a value, so you can drop it anywhere a value is allowed. In class we used it in three places:



One function, three ways to use it.

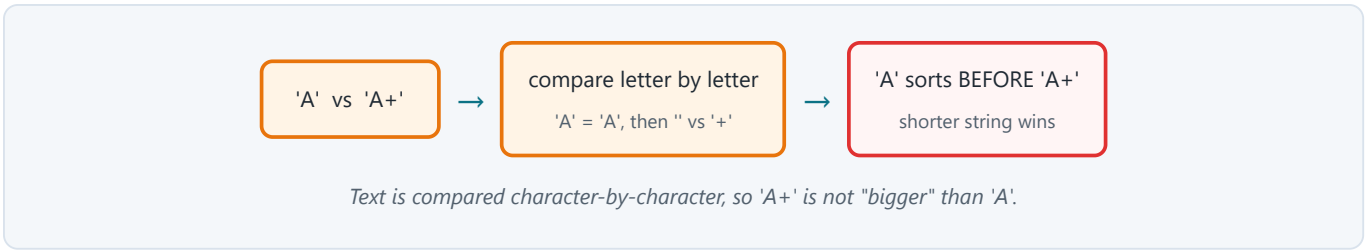
```
-- (1) In the SELECT list — grade appears as a new column
SELECT id, name, marks, getGrade(marks) AS grade
FROM student;

-- (2) In the WHERE clause — only the 'A' students
SELECT id, name, marks
FROM student
WHERE getGrade(marks) = 'A';

-- (3) In ORDER BY — sort by the computed grade
SELECT id, name, marks, getGrade(marks) AS grade
FROM student
ORDER BY grade;
```

🐞 The `A+` vs `A` sorting problem (and the fix)

When we ordered by the **grade text**, `A+` came **after** `A`. Why?



Text is compared character-by-character, so 'A+' is not "bigger" than 'A'.

The fix is to sort by a **number** instead of the text, using `CASE`:

```
SELECT id, name, marks, getGrade(marks) AS grade
FROM student
ORDER BY CASE getGrade(marks)
           WHEN 'A+' THEN 1
           WHEN 'A'  THEN 2
           WHEN 'B'  THEN 3
           WHEN 'C'  THEN 4
           WHEN 'D'  THEN 5
           ELSE 6
         END;
```

> 💡 **Simpler option**
>
> If your goal is just "best students first", you can simply `ORDER BY marks DESC` — the teacher pointed this out too. Use the `CASE` trick when the grade itself is the only ranking you have (for example when you only store a grade, not raw marks).

💼 Practical example — the annual package

The second big class example. HR gives you a **monthly salary**; you want the **annual package** (which includes an allowance, here HRA = 30% of salary).

```
DELIMITER $$
CREATE FUNCTION annual_package(monthly_salary DECIMAL(10,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
RETURN (monthly_salary + monthly_salary * 0.30) * 12;
$$
DELIMITER ;

SELECT annual_package(60000);     -- (60000 + 18000) * 12 = 936000.00
```

Then apply it to every employee — this is the real power of a function:

```
SELECT emp_id,
       emp_name,
       salary,
       annual_package(salary) AS annual_package
FROM employee;
```

Expected output (illustrative)

```
+--------+----------+--------+----------------+
| emp_id | emp_name | salary | annual_package |
+--------+----------+--------+----------------+
|    101 | Amit     |  80000 |     1248000.00 |
|    102 | Priya    |  50000 |      780000.00 |
|    103 | Rakesh   |  60000 |      936000.00 |
+--------+----------+--------+----------------+
```

ℹ️ Why package ≠ salary × 12

The teacher explained this well: a **CTC / package** (e.g. "12 LPA") is **not** the same as your monthly salary × 12. A package bundles **basic salary + HRA + travel allowance + bonus + other factors**. So we add the allowances *before* multiplying by 12. A 12 LPA package does **not** mean ₹1,00,000 per month.

🧪 Try it yourself (from the homework)

Add a **travelling allowance** of ₹5,000 per month to the function:

```sql
DELIMITER $$
CREATE FUNCTION annual_package2(monthly_salary DECIMAL(10,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
RETURN ((monthly_salary + monthly_salary * 0.30) + 5000) * 12;
$$
DELIMITER ;
```

⚠️ A real bug from class: name mismatch

In class the function was created as `annual_salary`, but it was being **called** as `annual_package` → an error. **The name you `CREATE` must be the name you `SELECT`.** When a function "doesn't exist", check the spelling first.

🏷️ Practical example — a discount function

Another practical example from class: a shop wants the price **after** a discount.

```sql
DELIMITER $$
CREATE FUNCTION discount_price(price DECIMAL(10,2), discount_pct DECIMAL(5,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
RETURN ROUND(price - price * discount_pct / 100, 2);
$$
DELIMITER ;

SELECT discount_price(10000, 10);    -- 9000.00   (10% off)
SELECT discount_price(4999, 5);      -- 4749.05   (5% off)
```

📃 Variables inside a function

Yes — you can `DECLARE` variables in a function, exactly like in a stored procedure. Declare them at the **top** of the `BEGIN … END` block.

```
DELIMITER $$
CREATE FUNCTION grade_with_var(marks INT)
RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
    DECLARE g VARCHAR(2);          -- local variable

    IF marks >= 90 THEN
        SET g = 'A+';
    ELSEIF marks >= 80 THEN
        SET g = 'A';
    ELSE
        SET g = 'F';
    END IF;

    RETURN g;                      -- return the variable
END $$
DELIMITER ;
```

> ℹ️ **Two styles, same result**
>
> The earlier `getGrade` used `RETURN` inside each branch. This version computes a value into a variable and returns it once at the end. Both are fine — pick the one that reads more clearly for your logic.

📌 One more thing: the `characteristics`

You already saw these fix the big error, but here is the full set in one place.

Characteristic	Meaning	Use when
`DETERMINISTIC`	same input → same output	pure calculations (`square` , `getGrade`)
`NOT DETERMINISTIC`	output may vary	uses `NOW()` , `RAND()` , reads changing data
`NO SQL`	body has no SQL statements	pure expressions
`READS SQL DATA`	reads tables, does not change them	functions that `SELECT`
`MODIFIES SQL DATA`	changes data	(rare in functions)
`SQL SECURITY DEFINER / INVOKER`	whose privileges it runs with	security-sensitive work

✅ Part A recap

- A **function must return one value**; you call it with `SELECT`, not `CALL`.
- Skeleton: `DELIMITER $$` → `CREATE FUNCTION` → `RETURNS <type>` → **characteristics** → `BEGIN … END` → `RETURN` → `$$` → `DELIMITER ;`.
- The **binary-logging error (1418)** is fixed by adding `DETERMINISTIC` / `NO SQL` / `READS SQL DATA` — or, temporarily, `SET GLOBAL log_bin_trust_function_creators = 1`.
- Use `DECIMAL(M,D)` for money, and give the **return type enough digits**.
- You can branch with `IF / ELSEIF / ELSE` and declare **variables**.
- Call functions in `SELECT`, `WHERE` and `ORDER BY` — and use `CASE` when text sorting gets in the way.

Next: making the database **react on its own**. ⚡

Part B — Triggers ⚡

Now for the topic the teacher called **the important one**: triggers. If functions are "a verb you teach the database", a trigger is **a reflex you give it**.

⚡ What is a trigger?

A **trigger** is a block of SQL that MySQL runs **automatically** when a specific change happens to a table — an `INSERT`, an `UPDATE`, or a `DELETE`.

> ℹ️ **The key difference from procedures and functions**
>
> Procedures and functions must be **called** by you (`CALL` / `SELECT`). A trigger is **never called by name**. It **fires on its own** the moment the event happens — whether the change came from you, an application, or a script.

Automatic vs manual



Nobody has to remember to run a trigger. That is its whole point.

Why triggers are useful (all mentioned in class):

- **Auditing** — keep a history of who changed what (the salary audit).
- **Enforcing rules** — reject bad data (salary cannot be negative).
- **Keeping data in sync** — reduce stock automatically when an order arrives.

🎛️ The six events you can trigger on

A trigger has two coordinates: **timing** (`BEFORE` or `AFTER`) and **event** (`INSERT` , `UPDATE` , `DELETE`). That gives **six** possible triggers per table.

Event	BEFORE the change	AFTER the change
INSERT	validate / clean the new row	log it, update other tables
UPDATE	validate / clean the new values	log old→new, recompute totals
DELETE	check preconditions	archive the removed row

Six events: BEFORE INSERT, AFTER INSERT, BEFORE UPDATE, AFTER UPDATE, BEFORE DELETE, AFTER DELETE.

💡 `BEFORE` or `AFTER` — how to choose

- Use `BEFORE` when you want to **inspect or change** the row *before* it is saved — validation, cleaning, and rejecting with `SIGNAL`.
- Use `AFTER` when the change must have **already happened** — logging, updating *other* tables, reading final values.

🔄 OLD and NEW — what is available when

Inside a trigger you get access to the affected row through two magic names:

- `NEW` — the row that is *about to be* (BEFORE) or *just* (AFTER) inserted/updated.
- `OLD` — the row as it was *before* an `UPDATE` or `DELETE`.

Event	OLD	NEW
INSERT	✗ not available	✓ available
UPDATE	✓ available	✓ available
DELETE	✓ available	✗ not available

Green = you can use it. Red = it does not exist for that event.

Why? (the reasoning from class) 🧠

- **INSERT:** you are creating a **brand-new** row, so there is no *old* version of it. Only `NEW` makes sense.
- **UPDATE:** you are changing an **existing** row, so both the *old* version and the *new* version exist. You have `OLD` **and** `NEW`.
- **DELETE:** you are removing a row, so there is no *new* version afterwards. Only `OLD` makes sense.

> 🛑 **Using the wrong one gives an error**
>
> Referring to `OLD.salary` inside an `INSERT` trigger — or `NEW.salary` inside a `DELETE` trigger — is an error, because that pseudo-row does not exist for that event. Keep the matrix above in your head and you will never hit it.

🧱 The general syntax

```
DELIMITER $$

CREATE TRIGGER trigger_name
    {BEFORE | AFTER} {INSERT | UPDATE | DELETE}     -- (1) pick ONE of the six events
    ON table_name                                   -- (2) which table it watches
    FOR EACH ROW                                    -- (3) fire once per affected row
BEGIN
    -- the body: your logic (INSERT / UPDATE / SET NEW.x = ... / SIGNAL …)
END $$

DELIMITER ;
```

#	Piece	Meaning
1	`{BEFORE | AFTER} {INSERT | UPDATE | DELETE}`	Choose **one** of the six events.
2	`ON table_name`	Which table's changes wake the trigger up.
3	`FOR EACH ROW`	Fire **once for every row** affected — if one statement changes 5 rows, the trigger runs 5 times.

> ℹ️ **No brackets, no parameters**
>
> In class the teacher pointed out that a trigger has **no parameter list** — there is nothing after the trigger name. The `OLD` / `NEW` pseudo-rows *are* its inputs. `FOR EACH ROW` is **required** in MySQL.

📝 Example 1 — the salary audit (`AFTER UPDATE`)

The main practical example. **Business rule:** "*Whenever an employee's salary is updated, record the old and new salary in an audit table — automatically.*"

Step 1 — the audit table

```
CREATE TABLE salary_audit (
  audit_id   INT AUTO_INCREMENT PRIMARY KEY,
  emp_id     INT,
  emp_name   VARCHAR(30),
  old_salary INT,
  new_salary INT,
  change_on  DATETIME
);
```

Step 2 — the trigger

```
DELIMITER $$
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
    INSERT INTO salary_audit (emp_id, emp_name, old_salary, new_salary, change_on)
    VALUES (NEW.emp_id, NEW.emp_name, OLD.salary, NEW.salary, NOW());
END $$
DELIMITER ;
```

Notice how we use **both** pseudo-rows: `OLD.salary` is the *before* value, `NEW.salary` is the *after* value.

Step 3 — watch it fire 🔥

```
UPDATE employee SET salary = 90000 WHERE emp_id = 101;
```

We never inserted into `salary_audit` — the trigger did.

```
SELECT * FROM employee WHERE emp_id = 101;
SELECT * FROM salary_audit;
```

Expected output

```
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
|    101 | Amit     |  90000 |
+--------+----------+--------+

+----------+--------+----------+------------+------------+---------------------+
| audit_id | emp_id | emp_name | old_salary | new_salary | change_on           |
+----------+--------+----------+------------+------------+---------------------+
|        1 |    101 | Amit     |      80000 |      90000 | 2026-10-16 12:06:40 |
+----------+--------+----------+------------+------------+---------------------+
```

What happens if you update *many* rows?

Because of `FOR EACH ROW`, every row that changes gets its own audit entry:

```
UPDATE employee SET salary = 50000;      -- no WHERE -> all rows change
SELECT * FROM salary_audit;              -- one audit row per employee
```



One UPDATE statement, three trigger firings — because of FOR EACH ROW.

> ⚠️ **The trigger fires for ANY update — not just salary**
>
> In class we also changed an employee's **name**, and the audit row still appeared. The trigger is attached to the **event** (`UPDATE`), not to a column. If you want to log **only** salary changes, add a guard:

```
BEGIN
    IF NEW.salary <> OLD.salary THEN
        INSERT INTO salary_audit (emp_id, emp_name, old_salary, new_salary, change_on)
        VALUES (NEW.emp_id, NEW.emp_name, OLD.salary, NEW.salary, NOW());
    END IF;
END
```

🚫 Example 2 — validation with `SIGNAL` (`BEFORE INSERT`)

Triggers can also **reject** bad data. Here: a salary must never be negative.

```
DELIMITER $$
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'salary cannot be negative';
    END IF;
END $$
DELIMITER ;
```

Now try a good insert and a bad insert:

```
-- a normal row: goes in fine
INSERT INTO employee VALUES (104, 'Rohit', 75000);

-- a bad row: rejected
INSERT INTO employee VALUES (105, 'Aniket', -25000);
```

Expected output

```
ERROR 1644 (45000): salary cannot be negative
```

> ℹ️ **What is `SIGNAL SQLSTATE '45000'` ?**
>
> `SIGNAL` is how you **raise your own error** from inside SQL. `SQLSTATE '45000'` is the conventional code for "unhandled user-defined exception", and `SET MESSAGE_TEXT = …` is the message the user sees. (The teacher said the full set of SQLSTATE codes will be covered in the next class.)

`BEFORE` vs `AFTER` — the crucial difference

This is the point the teacher demonstrated live:

Trigger timing	What happens on a bad value
BEFORE INSERT + SIGNAL	✓ The row is **NOT** inserted — the error stops it before it is saved.
AFTER INSERT + SIGNAL	✗ The row is already inserted; you only get the error message afterwards.

To PREVENT bad data, you must validate in a BEFORE trigger.

> 🧪 **The class experiment**
>
> The teacher dropped the `BEFORE` trigger, created an `AFTER INSERT` version, and inserted a negative salary. The message still appeared — but a **BEFORE** trigger is the one that actually **stops** the row. Lesson: **validate BEFORE, log AFTER.**

📦 Example 3 — stock management (`AFTER INSERT` on orders)

The second practical example: when an order arrives, **reduce the product's stock automatically**.

The tables

```sql
CREATE TABLE product (
  product_id   INT PRIMARY KEY,
  product_name VARCHAR(30),
  stock        INT
);

-- Remember: "orders", NOT "order" (ORDER is a reserved word!)
CREATE TABLE orders (
  order_id   INT,
  product_id INT,
  quantity   INT
);

INSERT INTO product VALUES
  (1, 'iPhone 17',   300),
  (2, 'Samsung Pro', 250),
  (3, 'Nokia',       500);
```

The trigger

```
DELIMITER $$
CREATE TRIGGER trg_stock_update
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    UPDATE product
       SET stock = stock - NEW.quantity
     WHERE product_id = NEW.product_id;
END $$
DELIMITER ;
```

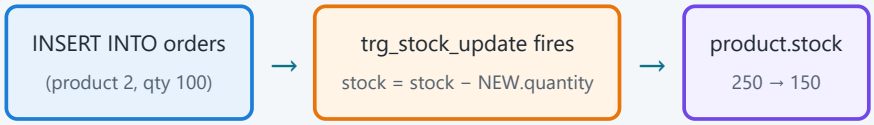
> ℹ️ **Notice what this trigger does NOT do**
>
> It does not touch the `orders` table at all — it only reads `NEW.quantity` and updates a **different** table (`product`). That is exactly why it works. (See the "same table rule" below.)

Watch it fire 🔥

```
INSERT INTO orders (order_id, product_id, quantity) VALUES (1, 2, 100);
SELECT * FROM product WHERE product_id = 2;
```

Expected output — stock drops from **250** → **150**:

```
+------------+--------------+-------+
| product_id | product_name | stock |
+------------+--------------+-------+
|          2 | Samsung Pro  |   150 |
+------------+--------------+-------+
```



One order in, stock out — with no manual UPDATE from you.

💡 Go further: refuse to oversell

Add a `BEFORE INSERT` guard so you can never order more than you have:

```
DELIMITER $$
CREATE TRIGGER trg_check_stock
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
    DECLARE avail INT;
    SELECT stock INTO avail FROM product WHERE product_id = NEW.product_id;
    IF NEW.quantity > avail THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Not enough stock for this order';
    END IF;
END $$
DELIMITER ;
```

🚧 The "same table" rule (error 1442)

This caused a lot of confusion in class, so let's make it crystal clear.

🛑 A trigger cannot run INSERT / UPDATE / DELETE on the table it is attached to

If your trigger is `ON employee`, then inside the body you **cannot** run `INSERT INTO employee …`, `UPDATE employee …`, or `DELETE FROM employee …`. MySQL raises **error 1442**:

```
ERROR 1442 (HY000): Can't update table 'employee' in stored function/trigger
because it is already used by statement which invoked this stored function/trigger
```

✅ Modify a DIFFERENT table	🚫 Modify the SAME table
trigger ON employee → write to salary_audit	trigger ON employee → write to employee

"Already used by the invoking statement" = the table is busy; you cannot re-enter it.

The class workaround: `employee_replica`

Because we could not write back into `employee`, the teacher created a **second table** and wrote there instead:

```
CREATE TABLE employee_replica (
  emp_id   INT,
  emp_name VARCHAR(30),
  salary   INT
);

-- Use the replica table whenever a trigger needs somewhere to write.
INSERT INTO employee_replica (emp_id, emp_name, salary)
SELECT emp_id, emp_name, salary FROM employee;
```

💡 How to change the row being saved (the *correct* way)

If you want to modify the **same row** that triggered the change, you do **not** run a separate `UPDATE`. You use a `BEFORE` trigger and assign to `NEW`:

```
CREATE TRIGGER trg_cap_salary
BEFORE INSERT ON employee
FOR EACH ROW
SET NEW.salary = LEAST(NEW.salary, 200000);
```

This edits the pending row in place — allowed, and the intended mechanism.

⚠️ A confusing thing we saw in class (worth knowing)

When experimenting with the replica table, a negative salary sometimes showed up as a **positive** value. The teacher noted this and said to check whether the column is declared `SIGNED` or `UNSIGNED`. A plain `INT` is *signed* (accepts negatives); an `UNSIGNED` column silently **cannot** hold a negative number. So if numbers look "flipped" or clamped, check your column's signedness first.

🔢 Multiple triggers on the same event: `FOLLOWS` and `PRECEDES`

A table **can** have more than one trigger for the same event (MySQL 5.7+). When it does, you control the order with `FOLLOWS` or `PRECEDES`.

```
-- Suppose trg_first already exists on employee AFTER UPDATE.
-- This trigger will run AFTER trg_first:
CREATE TRIGGER trg_second
AFTER UPDATE ON employee
FOR EACH ROW
FOLLOWS trg_first
BEGIN
    -- ...
END;
```

Keyword	Meaning
`FOLLOWS trg_first`	This trigger fires **after** trg_first.
`PRECEDES trg_first`	This trigger fires **before** trg_first.

Use one; if you specify neither, MySQL picks its own order.

> ⚠️ **The conflict we hit in class**
>
> Two triggers on the **same event** both fired, and the effects collided (one allowed the insert, the other blocked it). The teacher's lesson: either **drop the old trigger** before making a new one for the same event, or **order them deliberately** with `FOLLOWS` / `PRECEDES`. Do not let two triggers fight over the same job.

🧭 What triggers do *not* do

> 🛑 **The rules to remember**
>
> - **DML only.** Triggers fire for `INSERT`, `UPDATE`, `DELETE`. They do **not** fire for `CREATE`, `ALTER`, `TRUNCATE` (those are DDL).
> - **No `COMMIT` / `ROLLBACK` inside a trigger.** Transaction control is not allowed in the trigger body.
> - **No writing to the subject table** (the 1442 rule above).
> - **No dynamic SQL** (`PREPARE` / `EXECUTE`) inside a trigger.
> - Triggers are **not** fired by `TRUNCATE TABLE`, and (in some setups) not by `LOAD DATA`.

↩️ Triggers, transactions and `ROLLBACK` — the class demo

This was a genuinely important demonstration. You cannot put `COMMIT` / `ROLLBACK` **inside** a trigger — but the trigger's writes still belong to the **surrounding transaction**. So if you roll that transaction back, the trigger's writes disappear too.

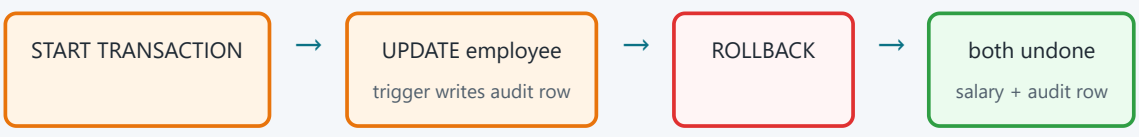
```
START TRANSACTION;

UPDATE employee SET salary = 95000 WHERE emp_id = 102;
-- → the AFTER UPDATE trigger fires and inserts a row into salary_audit

SELECT * FROM employee WHERE emp_id = 102;    -- shows 95000
SELECT * FROM salary_audit;                   -- shows the new audit row

ROLLBACK;                                      -- undo EVERYTHING

SELECT * FROM employee WHERE emp_id = 102;    -- back to 50000
SELECT * FROM salary_audit;                   -- the audit row is GONE
```



The trigger's INSERT is part of the same transaction — so it rolls back with the UPDATE.

> 💡 **Why this is reassuring**
>
> It means triggers are **transaction-safe**. You cannot half-apply a change: either the main change **and** the trigger's side-effect both happen, or neither does.

> ℹ️ **The transaction commands**
>
> `START TRANSACTION` , `SAVEPOINT` , `ROLLBACK` , `COMMIT` are the **transaction query language** statements. They work with the DML commands (`INSERT` / `UPDATE` / `DELETE`) — and therefore they also wrap whatever your triggers do.

🛠️ Managing triggers

```
-- List all triggers in the current database
SHOW TRIGGERS;

-- See the SQL that created a specific trigger
SHOW CREATE TRIGGER trg_salary_audit;

-- Remove a trigger
DROP TRIGGER IF EXISTS trg_salary_audit;
```

> 💡 **`SHOW TRIGGERS` is the first thing to check when data 'changes by itself'**
>
> Triggers run invisibly. If a table mutates in a way you did not expect, run `SHOW TRIGGERS;` — there is very often a trigger behind it.

✅ Part B recap

- A **trigger fires automatically** on `INSERT` / `UPDATE` / `DELETE` — no `CALL` .
- **Six events**: `{BEFORE, AFTER} × {INSERT, UPDATE, DELETE}` .
- `OLD` / `NEW` : INSERT → `NEW` only; **UPDATE → both**; DELETE → `OLD` only.
- **Salary audit** (`AFTER UPDATE` → audit table) and **stock update** (`AFTER INSERT` on orders → decrement `product.stock`) were our two main examples.
- `SIGNAL SQLSTATE '45000'` raises your own error — use it in a `BEFORE` trigger to *reject* bad data.
- **Cannot write to the subject table** (error 1442) — use a different table, or a `BEFORE` trigger with `NEW.col = …` .
- Multiple triggers on one event are ordered with `FOLLOWS` / `PRECEDES` .
- Triggers are **DML-only**, have **no** `COMMIT` / `ROLLBACK` **inside**, and their writes **roll back** with the surrounding transaction.

Now let's lock it all in with a cheat sheet and some practice. 🏁

Part C — Cheat Sheet, Gotchas & Practice 🏁

Everything from today, compressed into the pages you will actually come back to.

📋 Cheat sheet

Stored function — skeleton

```
DELIMITER $$
CREATE FUNCTION fn_name(p1 TYPE, p2 TYPE)
RETURNS return_type
DETERMINISTIC          -- or NO SQL / READS SQL DATA
BEGIN
    DECLARE v_x TYPE;
    -- logic: IF / ELSEIF / ELSE, SET, RETURN
    RETURN something;
END $$
DELIMITER ;
```

- Call it with `SELECT fn_name(...)` — never `CALL`.
- To change it: `DROP FUNCTION` then re-create (there is no `CREATE OR REPLACE`).

Trigger — skeleton

```
DELIMITER $$
CREATE TRIGGER trg_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
    -- NEW.* and OLD.* are available (see the matrix)
END $$
DELIMITER ;
```

`OLD` / `NEW` availability

Event	OLD	NEW
INSERT	✗	✓
UPDATE	✓	✓
DELETE	✓	✗

The single most useful table from today.

Function vs procedure (one more time)

	Procedure	Function
Returns a value	optional	**must**
Called with	`CALL`	`SELECT`
Usable in `SELECT` / `WHERE`	❌	✅
Parameters	`IN` / `OUT` / `INOUT`	`IN` only

Admin commands

Command	Shows
`SHOW FUNCTION STATUS WHERE Db = DATABASE();`	your functions
`SHOW CREATE FUNCTION fn_name;`	a function's definition
`SHOW CREATE PROCEDURE proc_name;`	a procedure's definition
`SHOW TRIGGERS;`	triggers in the database
`SHOW CREATE TRIGGER trg_name;`	a trigger's definition
`DROP FUNCTION / TRIGGER … ;`	remove one
`SHOW VARIABLES LIKE 'log_bin_trust_function_creators';`	the binary-logging setting

🐞 Errors we actually hit today — and exactly how to fix them

Error / problem	Why it happened	The fix
`ERROR 1418` … *none of DETERMINISTIC, NO SQL, or READS SQL DATA*	binary logging is on and the function does not declare its behaviour	add `DETERMINISTIC` / `NO SQL` / `READS SQL DATA`, **or** `SET GLOBAL log_bin_trust_function_creators = 1;`
Same 1418 error *again*	the keyword was typed `READ SQL DATA`	it must be `READS` SQL DATA (with an **s**)
`ERROR 1644 (45000)` salary cannot be negative	your own `SIGNAL` fired — this is working correctly!	nothing to fix — the row was correctly rejected
Cannot create table `order`	`ORDER` is a reserved keyword	name the table `orders` (or wrap it in backticks: `` `order` ``)
Two triggers fire and conflict	multiple triggers on the same event	drop the old one, or order them with `FOLLOWS` / `PRECEDES`
"Out of range value for column" / truncated result	the `RETURNS` type is too small	widen it, e.g. `DECIMAL(10,2)` → `DECIMAL(12,2)`
`ERROR 1442` … *Can't update table … in trigger*	the trigger writes to the table it is attached to	write to a **different** table; to change the row use `BEFORE` + `NEW.col = …`
Function "does not exist" when you call it	name mismatch (created `annual_salary`, called `annual_package`)	the name you create is the name you call — check spelling

Every one of these came up in today's session.

🧠 Practice exercises (do these — they mirror the class)

❓ Exercise 1 — a `cube` function

Write a function `cube(n INT)` that returns n × n × n.

❓ Exercise 2 — `net_salary`

Write `net_salary(basic DECIMAL(10,2), hra_pct DECIMAL(5,2))` that returns basic + (basic × hra_pct / 100).

❓ Exercise 3 — extend `getGrade`

Add an `'E'` grade for marks ≥ 40 (so only below 40 is `'F'`).

❓ Exercise 4 — audit DELETEs

Create a trigger that records deleted employees into a table, using the `OLD` values (remember: `NEW` is not available on `DELETE`).

❓ Exercise 5 — block negative stock

Create a `BEFORE INSERT` trigger on `orders` that raises an error if the order quantity is greater than the available stock.

❓ Exercise 6 — order two triggers

Create a second trigger on the same event as another one and make it run **after** the first using `FOLLOWS`.

❓ Exercise 7 — prove the rollback

`START TRANSACTION`, update a salary, check the audit row exists, then `ROLLBACK` and confirm both the salary and the audit row are gone.

🧩 Solutions

1. `cube(n)`

```
DELIMITER $$
CREATE FUNCTION cube(n INT)
RETURNS INT
DETERMINISTIC
RETURN n * n * n;
$$
DELIMITER ;

SELECT cube(3);   -- 27
```

2. `net_salary(basic, hra_pct)`

```
DELIMITER $$
CREATE FUNCTION net_salary(basic DECIMAL(10,2), hra_pct DECIMAL(5,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
RETURN basic + (basic * hra_pct / 100);
$$
DELIMITER ;

SELECT net_salary(50000, 30);   -- 65000.00
```

3. Extended `getGrade(marks)`

```
DELIMITER $$
CREATE FUNCTION getGrade(marks INT)
RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
    IF marks >= 90 THEN RETURN 'A+';
    ELSEIF marks >= 80 THEN RETURN 'A';
    ELSEIF marks >= 70 THEN RETURN 'B';
    ELSEIF marks >= 60 THEN RETURN 'C';
    ELSEIF marks >= 50 THEN RETURN 'D';
    ELSEIF marks >= 40 THEN RETURN 'E';
    ELSE      RETURN 'F';
    END IF;
END $$
DELIMITER ;
```

4. Audit DELETEs (`AFTER DELETE`)

```
CREATE TABLE employee_archive (
  emp_id     INT,
  emp_name   VARCHAR(30),
  salary     INT,
  deleted_on DATETIME
);

DELIMITER $$
CREATE TRIGGER trg_employee_delete
AFTER DELETE ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_archive (emp_id, emp_name, salary, deleted_on)
    VALUES (OLD.emp_id, OLD.emp_name, OLD.salary, NOW());
END $$
DELIMITER ;
```

5. Block negative stock (`BEFORE INSERT`)

```
DELIMITER $$
CREATE TRIGGER trg_check_stock
BEFORE INSERT ON orders
FOR EACH ROW
BEGIN
    DECLARE avail INT;
    SELECT stock INTO avail FROM product WHERE product_id = NEW.product_id;
    IF NEW.quantity > avail THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Not enough stock for this order';
    END IF;
END $$
DELIMITER ;
```

6. Ordering two triggers with `FOLLOWS`

```
CREATE TRIGGER trg_second
AFTER UPDATE ON employee
FOR EACH ROW
FOLLOWS trg_salary_audit       -- runs after the audit trigger
BEGIN
    -- additional logic here
END;
```

7. The rollback proof

```
START TRANSACTION;

UPDATE employee SET salary = 95000 WHERE emp_id = 102;

SELECT * FROM salary_audit;                    -- the audit row is here

ROLLBACK;

SELECT * FROM employee WHERE emp_id = 102;     -- salary restored
SELECT * FROM salary_audit;                    -- audit row gone
```

➡️ Next up

The teacher ended today by pointing at two remaining topics:

- **Cursors** 🎞️ — walking a result set one row at a time.
- **Error handling** — a fuller treatment of SQLSTATE codes and handlers.

Both come tomorrow. Everything we did today is the foundation for them.

🎓 Final words

Today you learned to **extend** MySQL and to make it **reflexive**:

- **Functions** let you name a calculation and reuse it everywhere — and the `DETERMINISTIC` error is just MySQL asking you to be honest about what your function does.
- **Triggers** let the database protect and maintain itself — validating, auditing, and syncing data without anyone remembering to ask.
- The habit that matters most: when you get an error, **read it**. Today's errors (1418, 1644, 1442, the `READS` typo) all told you exactly what was wrong.

Practise these — in the lab, on your own tables — until triggers stop feeling like magic and start feeling like tools. 🛠️

End of Day 8 notes.